Arrays

Computer Programming 1

<2025-10-05 Sun>

Outline



Outline



- Array: definition and use
- Array and functions
- Multi-dimensional array
- Array and pointers
- Array, pointers, and functions

Motivations



- If you need to enter and manage repetitive data (e.g., 10 temperature reading), the only possibility we have, so far, is defining one variable per reading, yielding to code which is clumsy and which works only for the case at hand

```
int main() {
  double reading_0, reading_1, reading_2, /* ..., */ reading_9;
  /* ... */
  sum = reading_0 + reading_1 + reading_2 + /* ... */ reading_9;
}
```

Motivations



- Some data (e.g., images, matrices) can be more easily represented by structures in which elements can be addressed by coordinates (e.g., x, y)

100	106	584	...									
397	387	10	...									
305	506	405	...									

Type and Array variables



- An array is a finite sequence of consecutive elements of the same type.
- The number of elements of an array (size) is decided when you write the program and fixed.
- For an array `A` of type `T` and size `dim`, the compiler allocates `dim` consecutive cells of `sizeof(T)` bytes (static allocation).
- The variable `A`, declared as a array, represents the address of the first element of the sequence.



This is how you access it in C++

```
int a[10] = { 42, 13, 99, ..., 7 }

a[0] /* is 42 */
a[1] /* is 13 */
[...]
```

Logical (and physical) Layout



This is how you access it in C++

```
int a[10] = { 42, 13, 99, ..., 7 }

a[0] /* is 42 */
a[1] /* is 13 */
[...]
```

This is what happens in memory

Address	Index	Value
1000	0	42
1004	1	13
1008	2	99
	...	
1028	10	7



Definition and initialization

```
type id[dim];
type id[dim] = { list_of_values };
type id[] = { list_of_values };
```

Examples

```
double a[];     // compile-time error
double a[25];   // array of 25 doubles
char d[] = {'a', 'e', 'i', 'o'}; // dimension 4
```

Definition of Arrays: constraints



- The size of the array must be computable at compile time:
  - explicitly, through the constant expression dim.
  - implicitly, via the size of the initialization list.
- If the initialization list is empty, the value is initialized to zero of type T.
- If there is no initialization, values are random (faster startup, no guarantees on initial data)

Example




```
#include <iostream>
using namespace std;

int main() {
  int zero_initialized[10] = {};
  int good_luck_with_this[10];

  for (int i = 0; i < 10; i++) {
    cout << zero_initialized[i] << " ";
  }
  cout << endl;
  for (int i = 0; i < 10; i++) {
    cout << good_luck_with_this[i] << " ";
  }
}
```

```
0 0 0 0 0 0 0 0 0 0
-1646108352 32767 1953135183 32514 1 0 -1646108040 32767 73728 0
```

Operations over Arrays



The only operation defined on arrays is selection with index (subscripting).

```
identifier[exp]
```

where `exp` is an expression evaluating to an integer and `identifier` is a variable declared as an Array.

Valid uses:

- Used input, assignments, passed by reference, etc.
- E.g., `cin >> a[i]; a[n+3] = 3 * 2; set_to_zero(a[3])`

Arrays and ”for” go together




```
#include <iostream>
using namespace std;

int main() {
  const int SIZE = 10;
  int ARRAY[SIZE];

  for (int i = 0; i < SIZE; i++) {
    // whatever operation you need to do on ARRAY
    cin >> ARRAY[i];
    cout << ARRAY[i];
  }
}
```

Example: Read and Reverse




```
#include <iostream>
using namespace std;

int main() {
  const int size = 4;
  double a[size];

  cout << "Enter " << size << " real numbers:\n";
  for (int i = 0; i < size; i++) {
    cout << i << ": ";
    cin >> a[i];
  }

  cout << "Here they are in reverse order:\n";
  for (int i = size - 1; i >= 0; i--)
    cout << "\t a[" << i << "] = " << a[i] << endl;
  return 0;
}
```

Do and dont's



- When looping DO NOT
  - start from 1 (instead of `0`)
  - end with `size` (instead of `size - 1`)
- DO Always use a symbolic constant you can pass and use around:

```
const int NUMBER_OF_GRADES = 10;
double grades[NUMBER_OF_GRADES];
```

Invalid operations on Arrays



- Arithmetic, assignment, and input operations are not defined on arrays.
- Comparison and output operations are defined, but give unpredictable results.
- For all these operations, it is necessary to write ad-hoc functions.
- If you put an array in a `struct`, however, assigning the struct, copies the array (since assignment is defined for `struct`).

Example 1




```
#include <iostream>
using namespace std;

int main() {
  int a[10] = {};
  int b[10] = {};

  // invalid array assignment
  // a = b;
  // invalid operands of types ‘int [10]’ and ‘int [10]’ to binary ‘operator+’
  // c = a + b;

  cout << "a == a: " << (a == a) << "\na == b: " << (a == b);
}
```

Examples of Invalid Operations




```
// This is ok
int a[4] = {1, 2, 3, 4};
int b[4] = {1, 2, 3, 4};

// This is not ok
// a++;        // error at compile time
// a=b;        // error at compile time
// cin >> a;   // error at compile time

cout << (a == b) << endl;   // compilation ok, but is return false
cout << (a <= b) << endl;   // compilation ok, but unpredictable result
cout << a << endl;          // compilation ok, but unpredictable result
```



Why are operations not allowed?

According to Why array can't be copied (look for `aioeu`):

1. B (grandparent of C++) had arrays (e.g., `a[10]`), but `a` would return the *address* of the arrays and could be used in operations (e.g., `a * 2` would return the address of the array `a` multiplied by two — does not make much sense, but to give the idea)
2. C was designed to be compatible with B, and implemeting array assignment (`a = b`), would make the language incompatible.

Into the wild: out of Bounds Exceptions



- The operator `[]` allows access outside the valid range `0..(dim - 1)` of an array.
- This sets the memory into a possibly inconsistent state and might have catastrophic effect in case of error (e.g., stack smashing)
- It is the responsibility of the programmer to manage the subscripting operation and ensure it never goes outside the array's range.
- Note: Compiler flags like `-fsanitize=bounds` can help identify out-of-bounds errors at run-time.

Living with no Boundaries is dangerous (example 1)




```
#include <iostream>
using namespace std;

int main() {
  int x[5];
  int b = 18;

  // Don't try this at home
  x[-1] = -1; // overflow
  x[5] = 5;   // overflow

  cout << "C++ won't complain ... x[5] is " << x[5] << endl;
  cout << "... but things might get nasty" << endl;
  cout << b << endl;
}
```

Living with no boundaries is dangerous (example 2)




```
int main() {
  short a[10];

  int i = 0;
  while (true) {
    a[i++] = 1;
  }
}
```



- If `p` is of type `*T` and `i` is an integer, `p+i` is the address after `i` positions of dimension `sizeof(T)`.
- Arithmetic operations like `p++` or `p += i` implicitly multiply the integer change by `sizeof(T)`.
- The difference between two pointers of the same type, `p2-p1`, is an integer representing the number of elements between the two addresses (implicitly divided by `sizeof(T)`).
- Layman version If `a` is a pointer to an integer:
  - `(a + 1)` means: move one integer down
  - `(a + 2)` means: move two integers down



The name of an array is implicitly equivalent to a constant pointer that points to the first element of the array itself.

Equivalences



- The address expressions `&p[i]` and `p+i` are equivalent.
- The value expressions `p[i]` and `*(p+i)` are equivalent.
- If `p` points to the first element of array `A`:
  - `p+i == &(A[i])`
  - `*(p+i) == A[i]`.

Array Size



When A is an array, its size can be calculated with:

```
int size = sizeof(A) / sizeof(T)
```

Address	Index	Value
1000	0	42
1004	1	13
1008	2	99
	...	
1028	10	7

Array Size (as pointer)



When A is an array, its size can be calculated with:

```
int size = &A[N] - &A[0]
```

Array Size (as pointer)



When A is an array, its size can be calculated with:

```
T A[10];
int size = *(&A + 1) - A;
```

- &A address of the array, pointer to array of `T[10]`
- `(&A + 1)` pointer to the first block of memory after the array A
- `*(&A + 1)` dereference, get back a pointer to an element of type T
- A pointer of type T
- `*(&A + 1) - A` pointer arithmetic, number of element of type T between the two address, hence, size of A

Example: Array and Memory




```
#include <iostream>
using namespace std;

int main() {
  short x[2];

  cout << "Array size using sizeof" << endl;
  cout << sizeof(x) / sizeof(short) << endl;

  cout << "Array size as difference between addresses of x[1] and x[0]" << endl;
  int size = *(&x + 1) - x;
  cout << size <<endl;

  cout << "Where is the array?" << endl;
  cout << "&x and &(x + 0) are " << &x << " x is " << x << endl;
  cout << "(&x + 1) is " << (&x+1) << " (x + 1) is " << (x + 1) << endl;
}
```

Example 1: Arrays and Pointer Arithmetic




```
#include <iostream>
using namespace std;

int main() {
  short a[3] = { 1, 2, 3 };

  for (int i = 0; i < 3; i++) {
    cout << "a[" << i << "] lives at: " << &a[i] << endl;
  }

  cout << "We can access using the index" << endl;
  for (int i = 0; i < 3; i++) {
    cout << a[i] << endl;
  }

  cout << "We can use pointer arithmetic" << endl;
  for (int i = 0; i < 3; i++) {
    cout << (a + i) << " -> " << *(a + i) << endl;
  }
}
```

Live with no boundaries (revisited)



- C++ does not check for boundaries (unless you use `-fsanitize=bounds`)
- Now we use pointers to understand how to nastily do a side effect

Example




```
#include <iostream>

using namespace std;

int main() {
  short a[3];
  short b = 256;

  for (int i = 0; i < 3; i++) {
    cout << "a[" << i << "] lives at: " << &a[i] << endl;
  }

  cout << "b    lives at: " << &b << endl;

  cout << "Address of b - address of a[0] is " << &b - &a[0] << endl;
  // computer outputs -4
  cout << "Hence a[-4] is: " << a[-4] << endl;

  // if you try to access b like this, you will most likely be fired
}
```

Examples



Various examples available in the zip uploaded with the slides.

Functions with Arrays



- A function can have a formal parameter of the type array of T:
- Arrays are different from scalar variables and we need to pay attention at what happens.

  type function_name(type array_var[], other_args) {...}

```
int print(int values[], int size);
int sum(double operand1[], doube operand2[], int size);
```



- When an array is passed to a function, only the address of the first element is copied.
- This is equivalent to passing the elements of the array by reference/pointer (pointer decay).

Consequences:

- The array's elements are not copied.
- We can omit size in the square brackets: it would not be used, in any case.
- We should pass the array size as an additional argument (we can pass a smaller size, if we want to)
- The array and its elements are implicitly passed by reference

Passing elements: piece of cake




```
#include <iostream>
using namespace std;

void by_value(int value) { value = -456; }
void by_reference(int &value) { value = 789; }

const int SIZE = 10;

int main() {
  int a[SIZE] = {}; // initialize to zero

  by_value(a[1]);
  cout << "element passed by value (unchanged): " << a[1] << "\n";

  by_reference(a[5]);
  cout << "element passed by reference (changed): " << a[5] << "\n";

  for (int i = 0; i < SIZE; i++) { cout << a[i] << " "; }
}
```

Arrays are references




```
#include <iostream>
using namespace std;

void f_modifying_array(int array[], int size) {
  for (int i = 0; i < size; i++) { array[i] = 456; }
}

const int SIZE = 10;

int main() {
  int a[SIZE] = {};           // initialize to zero
  f_modifying_array(a, SIZE); // set to 10

  for (int i = 0; i < SIZE; i++) { cout << a[i] << " "; }
}
```

Passing Constant Arrays



- You can prevent the array from being changed by using the `const` keyword
- This is efficient (no waste of CPU/memory) and ensures the array is used as read-only input

`void function(const type identifier[], ...)`

```
// Examples
int print(const int v[], ...);
void read(const char v[], ...);
```

Example (what happens with `const`)




```
#include <iostream>
using namespace std;

void try_to_modify(const int array[], int size) {
  array[0] = -10;  // c++ does not like this
}

int main() {
  int a[] = { 1, 2, 3 };
  try_to_modify(a, 3);
}
```

Equivalent Function Signatures (Simple Array)



The following are equivalent:

```
int f(int arr[dim]);
int f(int arr[]);
int f(int *arr);
int f(const int *arr); // nearly equivalent to the others
```



Inside a function, the argument *decays* to a pointer

Example




```
#include <iostream>
using namespace std;

void pointer_decay(int array[]) {
  cout << "Functions know nothing." << endl;
  cout << "Here the size of 'array' is: " << sizeof(array) << endl;

  int * b;
  cout << "A pointer, in fact, has size: " << sizeof(b) << endl;
}

int main() {
  int a[] = { 1, 2, 3 };

  cout << "Main knows about 'a'. Its size is: " << sizeof(a) << endl;
  cout << "In fact, an integer is " << sizeof(a[0]) << " (and 4 * 3 = 12)" << endl;
  pointer_decay(a);
}
```



Definition

- Arrays of arrays, generating multi-dimensional arrays (matrices).
- The size of all dimensions of an array, except the first one, must be specified.
  - Example of error: `int a[DIM1][]`.
- Missing elements in the initialization list are replaced with zeros.

Multi dimensional Arrays initialization



Syntax

```
type id[dim1][dim2]...[dimN];
type id[dim1][dim2]...[dimN] = { list_of_values };
type id[][dim2]...[dimN] = { list_of_values };
```

Example

```
int mat[2][3];
int mat[2][3] = { {1, 2, 3}, {4, 5, 6} };
int mat[][3] = { {1, 2, 3}, {4, 5, 6} };
```

Logical layout

1	2	3
4	5	6

Using multi-dimensional arrays



Nest ”for” loops.

```
// how you call them is arbitrary
const int ROWS = 10;
const int COLS = 20;

// empty with zeroes
int a[ROWS][COLS] = { {1, 2, 3}, {3, 4, 5} };

for (int i = 0; i < ROWS; i++) {
  for (int j = 0; j < COLS; j++) {
    cin >> a[i][j];
  }
 }
```

Example




```
#include <iostream>
#include <iomanip>
using namespace std;

const int DIM1 = 3;
const int DIM2 = 5;

void print(const int a[][DIM2]) {
  for (int i = 0; i < DIM1; i++) {
    for (int j = 0; j < DIM2; j++)
      cout << " " << setw(4)  << a[i][j];
    cout << endl;
  }
}

int main() {
  int a[DIM1][DIM2]= { {1,2,3,4,5}, {6,7,8,9,10}, {11,12,13,14,15} };
  print(a);
  return 0;
}
```

Memory Layout




```
int mat[2][3];
```

Logical layout

mat[0][0]	mat[0][1]	mat[0][2]
mat[1][0]	mat[1][1]	mat[1][2]

Physical Layout

mat[0][0]
mat[0][1]
mat[0][2]
mat[1][0]
mat[2][0]
mat[3][0]

Physical Layout and Addresses

mat, mat[0] —->	mat[0][0]	0x1000	42
	mat[0][1]	0x1004	99
	mat[0][2]	0x1008	109
mat[1] —->	mat[1][0]	0x10a2	12
	mat[2][0]	0x10b0	14
	mat[3][0]	0x10b4	123

Example




```
#include <iostream>
using namespace std;

int main() {
  int mat[3][2];

  cout << "mat lives here " << mat << endl;
  for (int i = 0; i < 3; i++) {
    cout << "mat[" << i << "] lives here " << &mat[i] << endl;
    for (int j = 0; j < 3; j++) {
      cout << "mat[" << i << "][" << j << "] lives here " << &(mat[i][j]) << endl;
    }
  }

  // sizes
  cout << sizeof(mat) / sizeof(int) << endl;
  cout << sizeof(mat[0]) / sizeof(int) << endl;
  cout << sizeof(mat[0][1]) << endl;
}
```

Example: sizes




```
#include <iostream>
using namespace std;

int main() {
  int mat[3][2];

  // sizes
  cout << sizeof(mat) / sizeof(int) << endl;
  cout << sizeof(mat[0]) / sizeof(int) << endl;
  cout << sizeof(mat[0][1]) << endl;
}
```

Why we need all dimensions but the last?



To allow for pointer arithmetic

```
#include <iostream>
using namespace std;

int main() {
  int mat[3][2];

  cout << mat << endl;
  // &mat + 1 moves by a full matrix
  cout << &mat + 1 << endl;

  // &mat[0] + 1 moves by one row
  cout << (&mat[0] + 1) << endl;
  cout << mat[1] << endl;
}
```

Example in function (pointer decay)




```
#include <iostream>
using namespace std;

void f(int d[][2]) {
  // here we do not know how many rows there are
  // (this is similar to what was happening with simple arrays)
  cout << d << endl;
  cout << &d + 1 << endl;

  // however, we know how to move row by row
  // (because we have passed the dimensions)
  cout << (&d[0] + 1) << endl;
  cout << d[1] << endl;
}

int main() {
  int mat[3][2];
  f(mat);
}
```

(Recall) Array, pointers and functions



Recall:

- In function calls, only the address to the first location of the array is copied.
- A formal array parameter can be specified using either the array notation or the pointer notation.

(Recall) Equivalent Function Signatures Simple Array



```
int f(int arr[dim]);
int f(int arr[]);
int f(int *arr);
int f(const int *arr);
```

Equivalent Function Signatures (Multi-dimensional Array)



The dimension of all but the first array dimension must be specified.

```
int f(int arr[dim1][dim2]...[dimN]);
int f(int arr[][dim2]...[dimN]);
int f(int *arr[dim2]...[dimN]);
int f(const int *arr[dim2]...[dimN]);
```

Returning an Array



- A C++ can return an array, in the form of a pointer
- You need to be sure that the array is still allocated when the function finishes (i.e., the array is either `static` or declared outside the function)

Example 1: Returning a Modified Array Passed as Argument (Valid)



- If an array is passed as an argument, the function receives a pointer to the original array, and can modify it and return its address. The array is allocated externally (in the caller's scope)

Example 1 (Valid)




```
const int dim = 10;

// v[] is the array passed to the function
int *times(int v[], int dim, int k) {
  for (int i=0; i<dim; i++)
    v[i] = v[i]*k;
  return v; // returning the address of the modified original array
}

int main() {
  int v[dim] = {1,2,3,4,5,6,7,8,9,10};
  int * w;
  w = times(v, dim, 4);
  // v and w both point to the same array: {4, 8, 12, 16, 20, 24, 28, 32, 36, 40}
}
```

Example 2: Incorrectly Returning a Local Array (Error)



- Returning a pointer to an array allocated locally on the stack inside the function results in undefined behavior (e.g., a segmentation fault).
- The local array is destroyed (released) when the function returns.

Example 2 (Error)




```
int *times(int v[], int dim, int k) {
  int w[dim]; // Local array, allocated on stack

  for (int i=0; i<dim; i++) {
    w[i] = v[i] * k;
  }

  return w; // ERROR: returning a pointer to a local variable
}
```

Example 3: Returning a Global/Static Array (Valid, but BAD idea)



It is valid to return a pointer to an array that is statically allocated internally to the function. This array is not destroyed when the function returns.

Example 3 (Valid, but BAD idea)




```
const int dim = 10;

// Note: "a" is statically allocated externally to the function
int *times(int k) {
  static int a[dim] = {1,2,3,4,5,6,7,8,9,10}; // Global/static array

  for (int i=0; i<dim; i++)
    a[i]=a[i] * k;
  return a;
}
```

Why it is a BAD idea



- The array is allocated once.
- Every time you call the function you are not allocating a new array: you are modifying the same data (so, calling the functions with the static array seems to be changing a variable outside the functon)

Why it is a BAD idea

```
>

#include <iostream>
using namespace std;

int* f(int i) {
  static int array[3];
  // cout << "The static array lives at " << array << endl;
  array[0] = i;
  return array;
}

int main() {
  int* a;
  a = f(4); // we bind 'a' to the array built in f
  cout << a[0] << endl;

  int* b;
  b = f(666);
  cout << b[0] << endl;
  cout << a[0] << endl; // .. alas, also "a" changed
```